Laboratorul 11. Obiecte și clase

Windows în Python: inventar de procese și jurnalul de evenimente

fișă de laborator · 2 × 90 min (cu pauză la mijloc) · două lucrări · se lucrează individual, la terminal

> **CE FACEȚI ASTĂZI.** Ținta e o mașină Windows. În Python, nu în PowerShell, modelați procesele și evenimentele ca obiecte: o clasă @dataclass cu câmpuri și o metodă e_suspect care răspunde singură (Lucrarea A). Apoi o clasă Inventar care se umple din JSON, interoghează procesele și scoate un raport de alarme (Lucrarea B).

> **CUM LUCRAȚI.** Fișa dă obiectivul, scheletul cu # TODO și comanda de rulare, nu rândul gata scris. După fiecare pas rulați și comparați cu caseta verde „Pe ecran trebuie să vezi". Explicația de programare (clase, @dataclass, self, metode) e în suportul de curs; aici o puneți în practică.

Fișierele de start

Din secțiunea sesiunii de pe ELSE descărcați start/ și puneți-l în depozitul de curs, sub arsenal/windows/start/. Conține două capturi de pe o mașină Windows de laborator:

Fișier	Ce conține (cum s-ar obține pe o mașină reală)
`procese.json`	lista proceselor: pid, nume, user, cale (Get-Process / tasklist, exportat)
`evenimente.json`	jurnalul de securitate: id, nivel, sursă, cont, mesaj (Get-WinEvent, exportat)

LUCRAREA A (nucleu). Procesul și evenimentul ca obiecte

Obiectiv: arsenal/windows/inventar.py definește clasele Proces și Eveniment ca @dataclass, cu o metodă e_suspect care răspunde singură. Le încarci din JSON.

A1. Clasa Proces, un tipar de date (15 min)

Un proces are patru câmpuri legate. Îl descrii ca @dataclass: numele câmpului și tipul lui.

```
from dataclasses import dataclass, asdict
import json

@dataclass
class Proces:
    pid: int
    nume: str
    user: str
    cale: str

p = Proces(3120, "update.exe", "ana", r"C:\Users\ana\AppData\Local\Temp\update.exe")
print(p.pid, p.nume)
print(p)
```

Pe ecran trebuie să vezi:

```
3120 update.exe
Proces(pid=3120, nume='update.exe', user='ana',
cale='C:\\Users\\ana\\AppData\\Local\\Temp\\update.exe')
```

De observat. print(p) arată toate câmpurile, nu o adresă de memorie: @dataclass a scris __repr__ pentru tine. Accesezi câmpurile cu punct, p.pid, nu cu p["pid"]. Calea e scrisă cu r"..." (șir brut), ca backslash-ul Windows să rămână backslash; în print(p) îl vezi dublat (\\), așa arată Python un backslash în interiorul unui șir.

A2. Metoda e_suspect pe Proces (18 min)

Un proces care rulează din folderul utilizatorului (Temp, Roaming) e un semn clasic. Pui întrebarea ca metodă: obiectul se uită la propria cale și răspunde.

```
CAI_SUSPECTE = ("\\Temp\\", "\\Roaming\\")

@dataclass
class Proces:
    pid: int; nume: str; user: str; cale: str
    def e_suspect(self):
        # TODO: True daca una din CAI_SUSPECTE e in self.cale
        ...

print(p.e_suspect())   # True (update.exe din Temp)
```

Pe ecran trebuie să vezi:

```
True
```

A3. Clasa Eveniment și ID-urile Windows (18 min)

Evenimentele Windows au ID-uri cu sens: 4625 logon eșuat, 1102 jurnal șters, 7045 serviciu instalat. Eveniment e tot o @dataclass, cu metoda lui e_suspect.

```
SUSPECTE_ID = {4625, 1102, 7045}

@dataclass
class Eveniment:
    id: int; nivel: str; sursa: str; cont: str; mesaj: str
    def e_suspect(self):
        # TODO: True daca self.id in SUSPECTE_ID SAU self.nivel == "critical"
        ...
```

A4. Le încarci din JSON (15 min)

Scrie o funcție care citește un fișier JSON și toarnă o listă de obiecte. Proces(**d) despachetează un dicționar în câmpuri.

```
def procese_din_json(cale):
    with open(cale, encoding="utf-8") as f:
        # TODO: intoarce [Proces(**d) for d in json.load(f)]
        ...

procese = procese_din_json("arsenal/windows/start/procese.json")
print("procese:", len(procese))
print("suspecte:", sum(1 for p in procese if p.e_suspect()))
```

Pe ecran trebuie să vezi:

```
procese: 7
suspecte: 2
```

> **DE REȚINUT.** Proces(**d) ia un dicționar din JSON și îi pune valorile în câmpurile obiectului, pe nume. Așa transformi date brute (dicționare) în obiecte cu comportament (metode).

LUCRAREA B (extindere). Clasa Inventar și raportul

Obiectiv: o clasă Inventar care ține lista de procese, știe s-o interogheze și scoate un raport JSON de alarme, comandat din terminal.

B1. Inventar, o clasă-container (18 min)

Inventar ține lista de obiecte Proces și are un constructor alternativ, din_json, care o umple. @classmethod primește clasa (cls) în loc de un obiect.

```
@dataclass
class Inventar:
    procese: list
    @classmethod
    def din_json(cls, cale):
        with open(cale, encoding="utf-8") as f:
            # TODO: return cls([Proces(**d) for d in json.load(f)])
            ...
    def __len__(self):
        return len(self.procese)

inv = Inventar.din_json("arsenal/windows/start/procese.json")
print("procese:", len(inv))
```

Pe ecran trebuie să vezi:

```
procese: 7
```

B2. Metodele de interogare (20 min)

Adaugă pe Inventar două metode: suspecte() (procesele care se declară suspecte) și pe_user() (procesele grupate pe utilizator, un dicționar de liste).

```
    def suspecte(self):
        # TODO: lista proceselor cu p.e_suspect()
        ...
    def pe_user(self):
        # TODO: dict user -> lista de nume (foloseste setdefault)
        ...

for p in inv.suspecte():
    print(f"  [{p.pid}] {p.nume}  {p.cale}")
print(inv.pe_user())
```

Pe ecran trebuie să vezi:

```
  [3120] update.exe  C:\Users\ana\AppData\Local\Temp\update.exe
  [3890] svch0st.exe  C:\Users\ana\AppData\Roaming\svch0st.exe
{'SYSTEM': ['System', 'svchost.exe'], 'ana': ['explorer.exe', 'chrome.exe', 'update.exe',
'powershell.exe', 'svch0st.exe']}
```

De observat. svch0st.exe (cu zero în loc de o) imită svchost.exe, dar rulează din Roaming: un truc clasic de malware, prins de metoda e_suspect.

B3. Raportul JSON, din obiecte (17 min)

asdict transformă un obiect înapoi în dicționar, ca să-l scrii în JSON. main() cu argparse scoate un raport de alarme: procese suspecte și evenimente de alarmă.

```
from dataclasses import asdict

def main():
    import argparse
    p = argparse.ArgumentParser()
    p.add_argument("capturi")
    p.add_argument("--json", action="store_true")
    a = p.parse_args()
    inv = Inventar.din_json(a.capturi + "/procese.json")
```

```
    ev = evenimente_din_json(a.capturi + "/evenimente.json")
    alarma = [e for e in ev if e.e_suspect()]
    # TODO: cu --json, json.dumps cu asdict pe suspecte si alarma;
    #       fara, un rezumat citibil
```

```
python -m arsenal.windows.inventar arsenal/windows/start
```

Pe ecran trebuie să vezi:

```
Procese: 7 | suspecte: 2
  [3120] update.exe  (ana)  C:\Users\ana\AppData\Local\Temp\update.exe
  [3890] svch0st.exe  (ana)  C:\Users\ana\AppData\Roaming\svch0st.exe
Evenimente: 8 | alarma: 5
  4625 [warning] administrator: An account failed to log on
  ...
  1102 [critical] administrator: The audit log was cleared
```

B4. Un test cu __eq__ (15 min)

@dataclass dă __eq__ gratis: compari obiecte după câmpuri. Scrie un test care verifică metoda e_suspect.

```
from arsenal.windows.inventar import Proces

def test_proces_din_temp_e_suspect():
    p = Proces(1, "x.exe", "ana", "C:\\Users\\ana\\AppData\\Local\\Temp\\x.exe")
    assert p.e_suspect() is True

def test_proces_din_windows_nu_e_suspect():
    p = Proces(2, "svchost.exe", "SYSTEM", "C:\\Windows\\System32\\svchost.exe")
    assert p.e_suspect() is False
```

```
python -m pytest -q arsenal/windows/test_inventar.py
```

Pe ecran trebuie să vezi:

```
2 passed
```

De observat. Rulezi din rădăcina depozitului, cu python -m pytest, nu doar pytest: așa folderul curent intră în calea de import și from arsenal.windows... se găsește. Cu pytest simplu primești ModuleNotFoundError: No module named 'arsenal'.

OPȚIONAL (dacă termini mai devreme sau acasă)

1. Proces pe o mașină reală: pe laptopul tău (sau pe VM-ul Windows de laborator), exportă procesele cu Get-Process | ConvertTo-Json și citește exportul cu inventarul tău.

2. O metodă explica() pe Eveniment care întoarce un text scurt pentru fiecare ID cunoscut (4625 = brute force, 1102 = jurnal șters, 7045 = serviciu nou).

3. O metodă scor() pe Inventar care dă un număr de risc (câte procese și evenimente de alarmă).

Ce predați și cum se notează

Predarea e prin git, în depozitul tehnici-prog-<nume>. Nimic pe e-mail, nicio captură de ecran.

```
git add arsenal/windows
git commit -m "windows: inventar procese + jurnal ca obiecte, in Python"
git push
```

Livrabil	Ce conține
`arsenal/windows/inventar.py`	clasele Proces, Eveniment (cu e_suspect) și Inventar (cu metode), raport JSON.
`arsenal/windows/test_inventar.py`	teste care trec cu pytest (Lab B).

Se punctează: @dataclass corect, metode cu self, întrebarea e_suspect ca metodă (nu funcție copiată), obiecte în loc de dicționare paralele, o clasă-container cu metode. Asistentul vă poate cere să explicați ce e self și de ce ați pus e_suspect pe clasă.